

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Create two test images
img1 = np.zeros((400, 500, 3), dtype=np.uint8)
img2 = np.zeros((400, 500, 3), dtype=np.uint8)

# Draw the same objects in both images
cv2.rectangle(img1, (100, 100), (300, 300), (255, 255, 255), 3)
cv2.circle(img1, (200, 200), 60, (255, 255, 255), 3)

cv2.rectangle(img2, (130, 80), (330, 280), (255, 255, 255), 3)
cv2.circle(img2, (230, 180), 60, (255, 255, 255), 3)

# Convert to grayscale
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

# Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and descriptors
kp1, des1 = sift.detectAndCompute(gray1, None)
kp2, des2 = sift.detectAndCompute(gray2, None)

# FLANN matcher
index_params = dict(algorithm=1, trees=5)
search_params = dict(checks=50)

flann = cv2.FlannBasedMatcher(index_params, search_params)

# Find two nearest matches
matches = flann.knnMatch(des1, des2, k=2)

# Lowe's ratio test
good_matches = []
```

```
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)

# Calculate accuracy
total_matches = len(matches)
good = len(good_matches)

accuracy = (good / total_matches) * 100 if total_matches > 0 else 0

# Draw good matches
result = cv2.drawMatches(
    img1, kp1,
    img2, kp2,
    good_matches, None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

# Display results
plt.figure(figsize=(15, 8))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(img1, cv2.COLOR_BGR2RGB))
plt.title("Image 1")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(img2, cv2.COLOR_BGR2RGB))
plt.title("Image 2")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title(f"Good Matches\nAccuracy: {accuracy:.2f}%")
plt.axis("off")

plt.tight_layout()
plt.show()
```

```
# Print results
print("FEATURE MATCHING ACCURACY EVALUATION")
print("-----")
print("Keypoints in Image 1 :", len(kp1))
print("Keypoints in Image 2 :", len(kp2))
print("Total Matches      :", total_matches)
print("Good Matches       :", good)
print("False Matches      :", total_matches - good)
print("Matching Accuracy   : {:.2f}%".format(accuracy))
```

Image 1

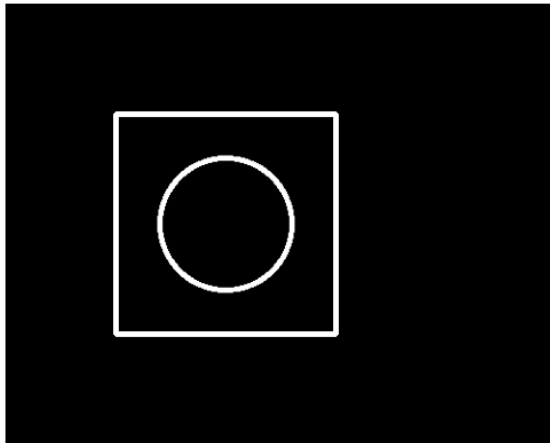
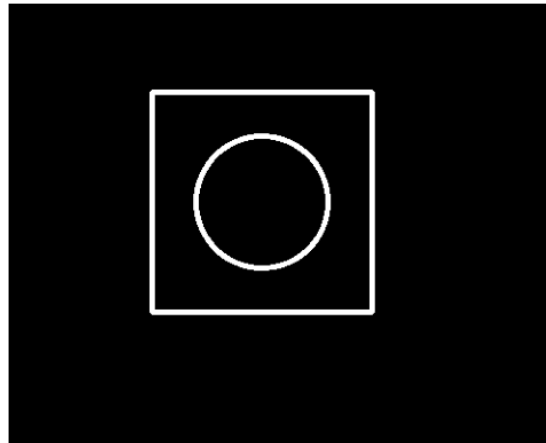
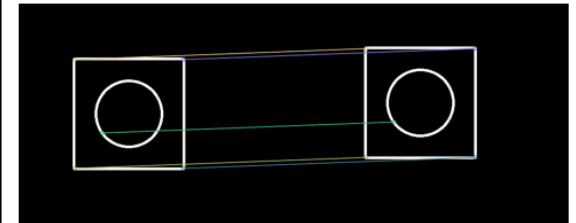


Image 2



Good Matches
Accuracy: 55.56%



FEATURE MATCHING ACCURACY EVALUATION

Keypoints in Image 1 : 18
Keypoints in Image 2 : 18
Total Matches : 18

